First we need to create a global variable to hold the current parsed token and also use the enumeration that appears in the description of the assignment for the token type:

```
typedef  enum {PLUS, MINUS, DIVIDE, MULT, REMAINDER, POWER, LPAREN,
RPAREN, NUMBER, ERROR, EOL} TokenType;

//Struct definition for Token
struct Token {
    TokenType type;
    int value;
};
// global variable that stores the currently parsed token
struct Token token;
```

The main function is going to ask the user for the input expression and then call the parse function. This function will first call he getTokenFuction to read in the first token. I used the implementation you explained in the tips and suggestions. First it discards the white space characters

```
int currentCharacter = 0;
while ((currentCharacter = getchar()) == ' ');
```

then it checks to see if the character is a digit. If a digit is found, the type of the token is set to NUMBER. The character is converted to the corresponding integer using a formula until we have the final value.

```
if (isdigit(currentCharacter)) {
        token.type = NUMBER;
        while (isdigit(currentCharacter)) {
            token.value = 10 * token.value + (currentCharacter -
'0');
            currentCharacter = getchar();
        }
```

When the while loop is exit is because a non-digit has been found so that character need to be read in again

```
ungetc(currentCharacter, stdin);
```

Print the value of the number

```
printf("%d\tNUMBER\n", token.value);
```

If the character is not a digit, use switch statements to assign the type and print it

```
switch (currentCharacter){
        case'+':
            token.type = PLUS;
            printf("+\tPLUS\n");
            break;
        case '-':
            token.type = MINUS;
            printf("-\tMINUS\n");
            break;
 ...
```

Now we have to create the functions and use recursion to follow the grammar.

```
int expr(void)
int term(void)
int power(void)
int factor(void)
int factor1(void)
```

We will use the match function to match the token types (`void match(TokenType tkType)`) and call for the next token

Start out by calling the expr function inside the parser function and store it in result

Each function calls a function in its grammar (expr calls term, term calls power, power calls factor, etc) and then stores the result. Each grammar function will also start checking the token types (expr will check for PLUS or MINUS, term will check for MULT, DIVIDE, or REMAINDER, power will check for POWER, etc). When the function finds the correct token type for its grammar, it will then call the match function on that type and add to the final result.

```
int expr(void) {
    int result = term();
    while (token.type == PLUS || token.type == MINUS) {
        if (token.type == PLUS) {
            match(PLUS);
            result += term();
        } else if (token.type == MINUS) {
            match(MINUS);
            result -= term();
        }
    }
    return result;
}
```

Do the same with the rest of the function. We also have to take into account some math errors that might happen such as division by 0 or modulo by zero.

You have to follow the grammar and utilize the match and getToken functions. The match function will check the current token read in (global variable) to the tokenType passed into this function. If there is a match, it will read the next token (getToken() which updates the global variable).

```
void match(TokenType tkType) {
    if (token.type == tkType) {
        token = getToken();
    } else {
        printf("Error\n");
    }
}
```